

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence

AI MINI PROJECT REPORT

AI-Based Fake Job Posting Detector

Student Name	Nandini Chaudhari
Roll No.	T006_____
Class	TY.BSc.IT
Subject	Artificial Intelligence_____
Academic Year	2026 - 2027
College Name	MVLU College

1. Introduction

Looking for a job today usually starts on a screen. Job portals, social media groups and messaging apps carry thousands of openings every day, and most of them are genuine. A small but harmful share is not. Fraudulent postings imitate real recruiters, borrow company names and describe attractive roles, and their real purpose is to collect money or personal details from the applicant. Students and fresh graduates, who are often eager and have little experience of hiring processes, are among the easiest targets.

The AI-Based Fake Job Posting Detector is an academic machine learning project built to help with this problem. It examines the information given in a posting, including the title, company profile, description, requirements, benefits and salary details, and uses Natural Language Processing (NLP) to turn that text into numerical features. Machine learning models then estimate whether the posting is likely to be fraudulent.

The warning signs the system learns from are the ones a careful reader would look for: requests for registration or security fees, unrealistic earning claims, urgent hiring language, requests for sensitive personal information and suspicious communication channels. The aim is to give the user a fast, preliminary risk assessment that supports further verification, and not to replace their own judgement.

2. Problem Statement

Online job platforms contain both legitimate and potentially fraudulent advertisements. Manually checking every posting is difficult and time-consuming. The aim is to develop an AI/ML-based system that analyses job-posting information and classifies it as Genuine or Potentially Fraudulent.

3. Objectives

- Collect and prepare a job-posting dataset suitable for training and testing.
- Pre-process the data and combine the relevant text fields into a single input.
- Convert the text into numerical features using TF-IDF.
- Train and apply three classification algorithms: Naive Bayes, Decision Tree and Logistic Regression.
- Compare the models using standard evaluation measures. ugh a Streamlit dashboard.
- Build a Streamlit dashboard that accepts a new job posting and returns a prediction with supporting information.

4. Technologies Used

The project uses a small set of widely adopted, open-source tools. Each was chosen for a specific role in the pipeline.

Technology	Role in the project
Python	Primary programming language for the whole project
Pandas	Loading the CSV dataset and preparing the data
NumPy	Numerical operations and array handling
Scikit-learn	TF-IDF vectorisation, model training and evaluation
Matplotlib / Seaborn	Charts and visualisation of model performance
Streamlit	Interactive web dashboard for user input and results
GitHub	Version control and hosting of the source code
Streamlit Community Cloud	Online deployment of the dashboard

5. Dataset

The project uses a file named `fake_job_postings.csv`. It is a synthetic, educational dataset created specifically for this academic project and is not a verified collection of real-world fraud cases. This is an important point, and it is discussed again in the Results and Limitations sections.

Property	Details
File name	<code>fake_job_postings.csv</code>
Source	Synthetic / educational dataset prepared for this project
Total records	300
Genuine postings (label 0)	240
Fraudulent postings (label 1)	60
Target variable	fraudulent (0 = Genuine, 1 = Fraudulent)
Attribute	Job_id, job_id, location, department, salary range Company_profile, description, requirements, telecommuting, has_company_logo

The dataset is imbalanced: genuine postings outnumber fraudulent ones four to one. This is deliberate, since real job boards also contain far more genuine than fraudulent advertisements.

6. Calculated Fields

The system follows a straightforward machine learning pipeline, from raw data to a final prediction shown to the user.

Data Loading → *Preprocessing* → *TF-IDF Features* → *Model Training* → *Evaluation* → *Prediction*

- 1. Load the data.** The CSV dataset is read into a Pandas DataFrame.
- 2. Prepare the text.** The relevant text columns, such as title, company profile, description, requirements and benefits, are combined into one text field for each posting.
- 3. Extract features.** TF-IDF converts the combined text into numerical vectors that the models can work with.
- 4. Train the models.** Naive Bayes, Decision Tree and Logistic Regression classifiers are trained on the feature vectors.
- 5. Evaluate.** Accuracy, precision, recall, F1-score and cross-validation are used to measure how well each model performs.
- 6. Accept new input.** New job-posting details are entered through the Streamlit interface.
- 7. Predict.** Each model gives its own prediction, and the system reports the fraud probability and how far the models agree.
- 8. Present the result.** Risk indicators and a final risk-oriented result are displayed to the user.

7. Algorithms Used

Three classifiers were used instead of one so that their results could be compared and so that the final output would not depend on a single model's opinion. All three work on the same TF-IDF features.

- **TF-IDF** :- TF-IDF (Term Frequency–Inverse Document Frequency) is the step that turns text into numbers. A word receives a higher weight when it appears often in a particular posting but is uncommon across the rest of the dataset. Common filler words therefore carry little weight, while distinctive terms, such as those linked to fees or urgent payment, stand out.
- **Naive Bayes**:- Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes that features contribute independently to the outcome. Although this assumption is simplified, the algorithm is fast and performs well on text classification with high-dimensional TF-IDF features. In this project it provides one independent prediction for each posting.
- **Decision Tree**:- A Decision Tree classifies a posting by passing it through a sequence of decision rules, each based on a feature. Its main strengths are that the rules are easy to read and that it can capture non-linear relationships. Its main weakness is a tendency to overfit, particularly on small datasets.
- **Logistic Regression**:- Logistic Regression is a supervised binary classification algorithm that estimates the probability that a sample belongs to a class. It handles TF-IDF features well and gives a clear probability score, which makes it a natural source for the fraud probability shown in the dashboard.

8. Implementation

The application is written in Python and uses Streamlit for the user interface. The dataset is kept in a Data folder and the required libraries are listed in requirements.txt, so the project can be set up on another machine with a single install command. The application combines the relevant text fields, applies TF-IDF, trains the three classifiers and presents the output through an interactive dashboard.

Project structure

```
project-folder/  
|-- app.py           (main Streamlit application)  
|-- requirements.txt (project dependencies)  
|-- Data/  
    |-- fake_job_postings.csv (dataset)
```

Loading the dataset

The dataset is loaded with a single Pandas call:

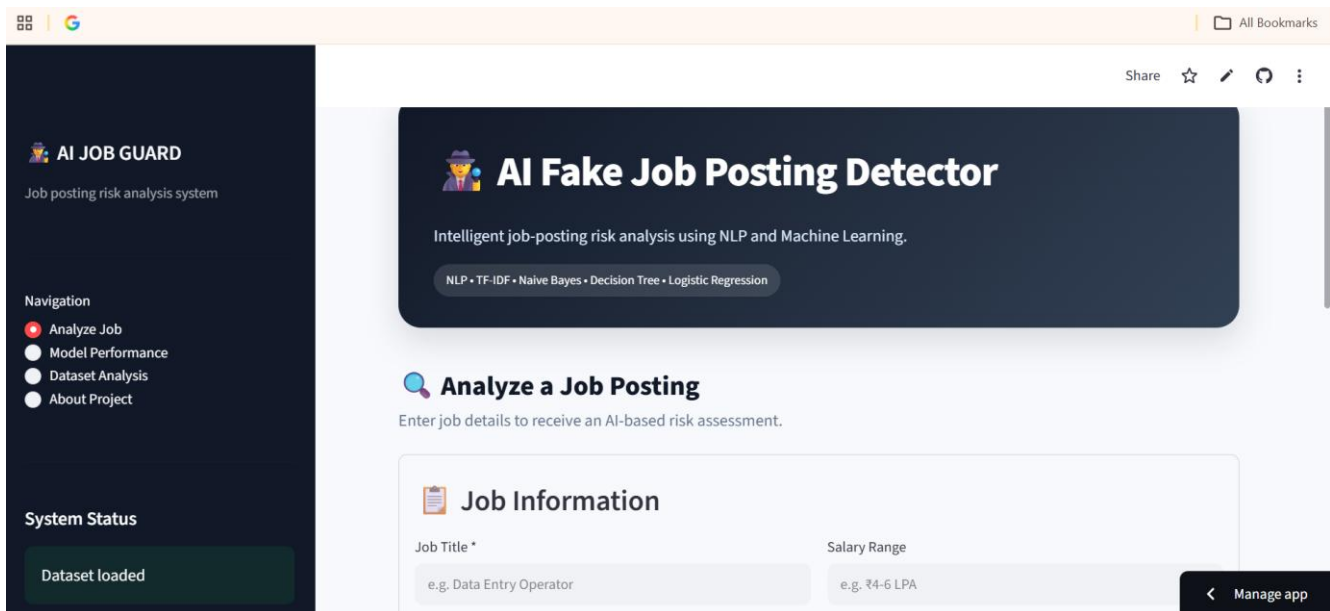
```
df = pd.read_csv("Data/fake_job_postings.csv")
```

Application screenshots

```

train_model.py > ...
1  import pandas as pd
2  import numpy as np
3
4  from sklearn.model_selection import (
5      train_test_split,
6      StratifiedKFold,
7      cross_val_score
8  )
9  from sklearn.feature_extraction.text import TfidfVectorizer
10 from sklearn.naive_bayes import MultinomialNB
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.pipeline import Pipeline
14 from sklearn.metrics import (
15     accuracy_score,
16     precision_score,
17     recall_score,
18     f1_score,
19     confusion_matrix,
20     classification_report
21 )
22 # =====
23 # 1. LOAD DATASET
24 # =====
25 DATA_PATH = "data/fake_job_postings.csv"
26 df = pd.read_csv(DATA_PATH)
27 print("Dataset loaded successfully!")
28 print("Shape:", df.shape)
29 # =====

```



9. Results

The finished system works as an interactive job-posting analysis dashboard. For every posting entered, it combines the predictions of Naive Bayes, Decision Tree and Logistic Regression and displays the fraud probability, the level of agreement between the models and a selection of risk indicators. This makes the output easier to trust and easier to question than a single label would be.

The performance of the three models, taken from the final training output, is recorded in Table.

The dataset is synthetic and contains strong fraud-related signals, so very high evaluation scores are likely. These figures show that the pipeline works as designed; they should not be read as the accuracy the system would achieve on real-world postings, which are far messier and more varied.

10. Advantages

- **Fast screening.** The system gives users a quick, preliminary risk check before they spend time on an application.
- **Text-based analysis.** NLP allows the whole text of a posting to be analysed, not just a few fields.
- **Multiple models.** Using three models reduces dependence on any single algorithm.
- **Explainable output.** Risk indicators show why a posting looks suspicious, instead of returning an unexplained label.
- **Ease of use.** The Streamlit dashboard needs no technical knowledge to operate.

11. Video Demonstration

Video Link: <https://youtu.be/rGdEO1GD438>

12. GitHub Repository

GitHub Link: <https://github.com/dibyanjalyyadav-pixel/AI-Practical-/tree/main/AI-Fake-Job-Detector>

13. Limitations

- **Synthetic data.** The dataset is synthetic and intended for academic demonstration, so results may not carry over to real postings.
- **Optimistic scores.** Because the data contains strong signals, measured performance is likely to be overly optimistic.
- **No guarantees.** The system cannot guarantee that a posting is fraudulent or genuine; it only estimates risk.
- **No independent verification.** Companies and recruiters are not checked against any external source.

14. Applications

- Preliminary screening of online job advertisements.
- Job-search assistance platforms.
- Student placement and career-guidance portals.
- Recruitment websites, as an additional risk-screening layer.
- Classroom demonstrations of NLP and machine learning.

15. Conclusion

This project demonstrates how Artificial Intelligence, Natural Language Processing and Machine Learning can be applied to the analysis of job advertisements. TF-IDF provides the text representation, Naive Bayes, Decision Tree and Logistic Regression perform the classification, and a Streamlit dashboard makes the whole system interactive and approachable.

The result is a working academic prototype that gives a risk-oriented assessment of a job posting. Its output should be treated as a decision-support signal and not as proof of fraud. Used sensibly, it can prompt a job seeker to pause and verify a company before sharing money or personal information, which is exactly the habit that protects people from scams.

16. Future Scope

Several improvements could turn this prototype into a more reliable tool:

- Training on a larger, real-world dataset whose labels have been independently verified.
- Using advanced NLP techniques, including transformer-based models.
- Connecting to company and recruiter verification sources.
- Analysing the URLs and domains that appear in job postings.
- Adding multilingual support for Indian languages.
- Retraining the model regularly as newly verified data becomes available.

17. References

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [2] Scikit-learn documentation – machine learning algorithms and TF-IDF text feature extraction. <https://scikit-learn.org>
- [3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- [4] Pandas documentation – data handling and CSV processing. <https://pandas.pydata.org>
- [5] Streamlit documentation – dashboard development and deployment. <https://docs.streamlit.io>
- [6] Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.
- [7] GitHub – project source-code repository: [insert repository link].
- [8] Course notes and classroom material on Artificial Intelligence, NLP and Machine Learning.